ngee ann polytechnic

School of InfoComm Technology

Machine Learning

Specialist Diploma in Data Analytics (SDDA),

Oct 2025 Semester

ASSIGNMENT 1

(40% of Machine Learning Module)

Timeline:

Presentation: 15/16th Dec 2025

Report & Codes Submission: 20th Dec 2025 (Sat), 23:59

Student Name	:	Tan Yong Cong, Elgin
Student Number	:	9278674D

Penalty for late submission:

10% of the marks will be deducted every day after the deadline.

NO submission will be accepted after 27th Dec 2025, 23:59.

.

1.0 Table Content:

4.4 Summary
4.4.1 Model Ranking (Best to Worst)
4.4.2 Final Selection:

5.0 Conclusion
5.1 Suggest possible further improvement(s) to the current solution.

6.0 Reflection
6.1 Suggest possible further improvement(s) to the current solution.
6.2 With reference to the module leaning objectives stated, reflect on the skills learnt and the skills you could have learnt better.

2.0 Introduction

2.1 A brief introduction on solving classification and regression problems using Machine Learning Models

2.2 What is Machine Learning?

Machine Learning (ML) is a subset of Artificial Intelligence that enables computers to learn patterns from data and make predictions or decisions without being explicitly programmed. ML models improve their performance automatically as they are exposed to more data.

2.3 What is Supervised Learning?

Supervised Learning consists of two main types of problems: regression and classification.

- Linear Regression → predicts continuous numerical values
- Logistic Regression → predicts categories (classification)

Supervised Learning is a type of Machine Learning where the model is trained using labeled data.
Common models in supervised learning include:

- Logistic Regression
- Linear Regression
- Decision Tree
- Random Forest
- XGBoost
- Support Vector Machines (SVM)

2.4 What is Classification in Supervised Machine Learning?

Classification is a supervised learning task where the model predicts a category or class label.
Examples:

- True or False
- Binary value: 0 or 1
- Promoted or Not Promoted
- Yes or No

Classification models are commonly evaluated using accuracy metrics:

- Train accuracy
- Test accuracy

2.5 What is Regression in Supervised Machine Learning?

Regression is a supervised learning task where the model predicts continuous numerical values, such as:

- Price
- Salary
- Temperature
- Stock prices

Regression models are evaluated using metrics such as:

- R^2 (Coefficient of Determination)
- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)

2.6 What is Train-Test Split?

Train-test split is a method used to divide the dataset into two parts, e.g., 70% training and 30% testing.

- The training set is used to train the model and learn data patterns.
- The testing set is used to evaluate the model's performance on unseen data and check how well it can predict future outcomes.

2.7 Why is model improvement necessary?

After evaluating the train-test results, you may want to improve the model to increase accuracy or reduce errors. This can involve tuning parameters, trying different algorithms, or improving data preprocessing. These metrics help assess how well the model performs on unseen data.

2.8 Technique Uses?

- Feature Selection → choosing relevant features to improve performance
- P-Value → statistical significance of features (common in regression/logistic regression)
- Feature Importance → how much each feature contributes to model predictions
- Hyperparameter Tuning → optimizing model parameters (e.g., GridSearchCV)
- K-Fold Cross-Validation → robust model evaluation technique

3.0 HR Analytics

3.1 Problem understanding and the approaches

3.1.1 Classification Problem

This project focuses on solving classification problems using an HR Analytics dataset to predict whether an employee "is promoted" or "not promoted". The target variable (y) is a binary label: 1 = promoted, 0 = not promoted.

And all the features are linked together as it determines the staff promotion next in-line. This is the chance to see the accuracy of the model to prove the model's performance capability.

The dataset is generally clean, however, the feature, "previous_year_rating", contains missing values (NaN) that must be handle before model training.

3.1.2 Handling Missing Values

```
# count how many null value -> sum()
nan_sum = data_01.isnull().sum()
print(nan_sum)
```

```
employee_id              0
department               0
region                   0
education                0
gender                   0
recruitment_channel      0
no_of_trainings          0
age                      0
previous_year_rating    679
length_of_service        0
KPIs_met >80%            0
awards_won?              0
avg_training_score       0
is_promoted              0
dtype: int64
```

```
# count how many null value -> with percentage
nan_percentage = data_01.isnull().mean() * 100
print(nan_percentage)
```

```
employee_id             0.000000
department              0.000000
region                  0.000000
education               0.000000
gender                  0.000000
recruitment_channel     0.000000
no_of_trainings         0.000000
age                     0.000000
previous_year_rating    7.272922
length_of_service       0.000000
KPIs_met >80%           0.000000
awards_won?             0.000000
avg_training_score      0.000000
is_promoted             0.000000
dtype: float64
```

Table 1: NaN value of "previous_year_rating"

Table 1: Present the number of NaN values in the "previous_year_rating" column. Because this feature reflects an employee's past performance, it is an important factor for predicting promotions.

3.1.3 Imputation Method

Therefore, imputation is required.

To determine the most suitable imputation method, the following steps were taken:

```
## need to remove or imputation?

checking_value = data_01['previous_year_rating'].value_counts().sort_index(ascending = False)
print(checking_value)

def explanation():
    """
Upon seeing the result:
"previous_year_rating"
3.0 had the most values follow by 5.0.
The least value is 2.0.
    """

print(explanation.__doc__)
```

```
previous_year_rating
5.0    2855
4.0    1658
3.0    2960
2.0     535
1.0     649
Name: count, dtype: int64

Upon seeing the result:
"previous_year_rating"
3.0 had the most values follow by 5.0.
The least value is 2.0.
```

Table 2: previous_year_rating, value_counts()

Table 2: I used value_counts() to observe the distribution of rating values to pick the highest value to replace the NaN value.

```
# mean testing
# # Replace all null values in the Age column with the mean value
data_02['previous_year_rating'] = data_02['previous_year_rating'].fillna(data_02['previous_year_rating'].mean())
data_02['previous_year_rating'] = data_02['previous_year_rating'].astype(int)
data_02['previous_year_rating'].isnull().any()

data_02.head()
```

	employee_id	department	region	education	gender	recruitment_channel	no_of_trainings	age	previous_year_rating	length_of_service	KPIs_met >80%	awards_
0	49017	7	7	1	0	1	1	35	5	3	1	
1	58304	7	28	1	1	1	1	33	5	6	1	
2	17673	7	4	2	1	0	1	50	4	17	1	
3	77981	1	22	1	1	0	1	27	3	1	1	
4	16502	7	22	1	1	1	1	27	3	1	0	

Table 3: Mean

```
# median testing
# # Replace all null values in the Age column with the mean value
data_02['previous_year_rating'] = data_02['previous_year_rating'].fillna(data_02['previous_year_rating'].median())
data_02['previous_year_rating'] = data_02['previous_year_rating'].astype(int)
data_02['previous_year_rating'].isnull().any()

data_02.head()
```

	employee_id	department	region	education	gender	recruitment_channel	no_of_trainings	age	previous_year_rating	length_of_service	KPIs_met >80%	awards_
0	49017	7	7	1	0	1	1	35	5	3	1	
1	58304	7	28	1	1	1	1	33	5	6	1	
2	17673	7	4	2	1	0	1	50	4	17	1	
3	77981	1	22	1	1	0	1	27	3	1	1	
4	16502	7	22	1	1	1	1	27	3	1	0	

Table 4: Median

```
# mode testing
data_03['previous_year_rating'] = data_03['previous_year_rating'].fillna('3.0')
data_03['previous_year_rating'] = data_03['previous_year_rating'].astype(float).astype(int)
data_03['previous_year_rating'].isnull().any()
data_03.head()
```

	employee_id	department	region	education	gender	recruitment_channel	no_of_trainings	age	previous_year_rating	length_of_service	KPIs_met >80%	awards_
0	49017	7	7	1	0	1	1	35	5	3	1	
1	58304	7	28	1	1	1	1	33	5	6	1	
2	17673	7	4	2	1	0	1	50	4	17	1	
3	77981	1	22	1	1	0	1	27	3	1	1	
4	16502	7	22	1	1	1	1	27	3	1	0	

Table 5: Mode

Mean, Median, and Mode were calculated to understand the central tendency (Table 3-5).

3.1.4 Insight from Rating Distribution

Based on the distribution previous_year_rating (Table 2):

- Rating 5.0 has one of the highest frequencies.
- Rating 4.0 is also highly common.
- Rating 3.0 has the largest count.
- Rating 2.0 shows lower frequency.
- Rating 1.0 has the lowest frequency.

This distribution provides insight into employee performance levels:

- Rating 5.0 and Rating 4.0 combined account for 4, 513 employees, indicating strong performers who are likely to be promoted.
- Rating 3.0 includes 2, 960 employees, representing average performers who are competing for promotion.
- Rating 2.0 and Rating 1.0 combined total 1, 184 employees, reflecting those with weaker performance records.

All three statistics (mean, median, and mode) produced similar values. However, mode imputation was chosen because it best represents the most frequently assigned performance rating and aligns with typical HR evaluation practices.

3.2 Build the model(s)

3.2.1 Model Selection and Training Approach

After completing all necessary data preprocessing steps, including handling missing values and preparing the feature set, I proceeded with building predictive models to analyse employee promotion outcomes.

Since the target variable (is_promoted) is binary (1 = promoted, 0 = not promoted), this is a classification problem.

1.2.1 Classification Model

Model 1

Model - Classification Problem

Logistic Classification

```
# Split both Inputs (X) and Output (y) into training set (80%) and testing set (20%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.3, random_state = 55)

# Random state ensure consistent random samples are selected
```

Table 6: Logistic Classification, sample of the train-test split

1.2.2 Classification Model

Model 2

Model - Classification Problem

Decision Tree

```
# Split both Inputs (X) and Output (y) into training set (80%) and testing set (20%)
X_train_02, X_test_02, y_train_02, y_test_02 = train_test_split(X, y, test_size = 0.3, random_state = 55)

# Random state ensure consistent random samples are selected
```

Table 7: Decision Tree, sample train-test split

I performed quite a few different train-test split for my model selection. Shows in Table 6 & 7. Why I do this is because I don't want to mix it and separate of it and shows this is for which model selection.

3.2.2 Machine Learning Models Used

To ensure a comprehensive comparison, I selected five widely used classification algorithms commonly applied in HR analytics and promotion prediction studies:

- Logistic Regression
- Decision Tree Classifier
- Random Forest Classifier
- XGBoost Classifier
- Support Vector Classifier (SVC)

I ran all 5 models to compare accuracy and predictive performance, together with the evaluation listed below.

In addition, I also perform pre-check results to compare post-check results to ensure consistency and stability in the evaluation process.

3.2.3 Evaluation Techniques

- Confusion Metrics
- K-Fold Cross Validation

So below is the sample of the result of individual technique used:

```
The Logistic Regression, train accuracy is: 0.6610558530986993
The Logistic Regression, test accuracy is: 0.6626204926811853
The Logistic Regression, Confusion Matrix train Accuracy: 0.6610558530986993
The Logistic Regression, Confusion Matrix train Precision: 0.6473291211947156
The Logistic Regression, Confusion Matrix train Recall: 0.6954643628509719
The Logistic Regression, Confusion Matrix train F1 score: 0.6705339878030642

The Logistic Regression, k-fold average train accuracy is: 0.6536795150360106
The Logistic Regression, k-fold average test accuracy is: 0.6430999817637336
```

Table 8: Sample of Individual Technique of Accuracy

1.2.4 Pre-check of Train and Test accuracy result

Accuracy	Logistic Regression	Decision Tree	Random Forest	XGBoost	SVC
Train	0.7161438408569243	0.6944146901300688	0.7666411629686305	0.8579954093343535	0.5066564651874522
Test	0.7183148875401643	0.6808282756158515	0.7661549446626205	0.8111388789717958	0.4844698322027847

Confusion Matrix: Train	Logistic Regression	Decision Tree	Random Forest	XGBoost	SVC
Accuracy	0.7161438408569243	0.6944146901300688	0.7666411629686305	0.8579954093343535	0.5066564651874522
Precision	0.7150837988826816	0.6872366044551474	0.7202258726899384	0.8021949307551607	0.5063197026022305
Recall	0.710891700092564	0.7044122184510954	0.8657821659981487	0.947238506633755	0.21012033323048443
F1	0.712981587498066	0.6957184214536035	0.7863247863247863	0.8687040181097906	0.29699084169210643

Confusion Matrix: Test	Logistic Regression	Decision Tree	Random Forest	XGBoost	SVC
Accuracy	0.7183148875401643	0.6808282756158515	0.7661549446626205	0.8111388789717958	0.4844698322027847
Precision	0.7285100286532952	0.6870175438596491	0.724157955865273	0.7653664302600472	0.48576214405360135
Recall	0.7126839523475823	0.6860546601261388	0.8738612473721094	0.9074982480728802	0.20322354590049055
F1	0.720510095642933	0.6865357643758766	0.7919974595109559	0.8303943571657583	0.2865612648221344

K-Fold: Avg	Logistic Regression	Decision Tree	Random Forest	XGBoost	SVC
Train	0.7197408420435605	0.6903384695305136	0.7684233342127643	0.8566569314925185	0.5040701423050304
Test	0.7172240961865558	0.6903383916989434	0.7617822910943939	0.8167304553675985	0.4896089983931441

Table 9: Model Accuracy Performance

3.2.4 Normal Run Accuracy (Train/Test Split)

In this Table 9, I will explain the normal run accuracy follow by confusion and k-fold.
Out of the 5 model only two model did the best.

Normal Run Accuracy (Train/ Test split)

1. XGBoost – Best Performance
- Train: 0.8579954093343535
- Test: 0.8111388789717958

2. Random Forest:
- Train: 0.7666411629686305
- Test: 0.7661549446626205

Confusion Matrix Result:
Based on Accuracy, Precision, Recall, and F1-score:
XGBoost shows the strongest overall performance.

K-Fold Cross Validation Result:

3. XGBoost Best Performance in K-Fold:
- Train Average Accuracy: 0.8566569314925185
- Test Average Accuracy: 0.8167304553675985

4. Random Forest accuracy:
- Train Average Accuracy: 0.7684233342127643
- Test Average Accuracy: 0.7617822910943939

3.2.5 Comparison Summary

XGBoost

- The train accuracy from confusion matrix is slightly higher than K-Fold.
- The test accuracy from k-fold is higher than confusion matrix.
- This indicates good generalisation and stability.

Random Forest

- K-fold accuracy is slightly higher than confusion matrix for training.
- This suggests Random Forest was more stable under k-fold validation than under a single train-test split.

3.3 Evaluate and improve the model(s)

After obtaining the pre-check model results, I performed additional post-check evaluation to identify whether any model showed improvement, stability, or significant performance changes.

To do this, I applied multiple techniques that fall into three main categories: Model Improvement, Hyperparameter Tuning, and Evaluation Techniques.

3.3.1 Techniques for Model Improvement

These methods help improve the model and understand which features matter the most.

Feature selection

- P-value (Logistic Regression)
- Feature Important (Tree-based models)

Hyperparameter Tuning

- Grid Search

Evaluation Techniques

- Confusion Matrix
- K-fold Cross validation

Logit Regression Results

Dep. Variable:	y	**No. Observations:**	6535
Model:	Logit	**Df Residuals:**	6522
Method:	MLE	**Df Model:**	12
Date:	Tue, 09 Dec 2025	**Pseudo R-squ.:**	0.1970
Time:	13:24:40	**Log-Likelihood:**	-3637.0
converged:	True	**LL-Null:**	-4529.5
Covariance Type:	nonrobust	**LLR p-value:**	0.000

	coef	std err	z	P>|z|	[0.025	0.975]
x1	-3.726e-06	1.25e-06	-2.991	0.003	-6.17e-06	-1.28e-06
x2	0.0047	0.011	0.426	0.670	-0.017	0.026
x3	-0.0209	0.003	-7.222	0.000	-0.027	-0.015
x4	0.1335	0.066	2.038	0.042	0.005	0.262
x5	-0.1464	0.062	-2.376	0.017	-0.267	-0.026
x6	-0.1140	0.052	-2.202	0.028	-0.216	-0.013
x7	-0.4694	0.055	-8.600	0.000	-0.576	-0.362
x8	-0.0795	0.005	-16.090	0.000	-0.089	-0.070
x9	0.2138	0.025	8.622	0.000	0.165	0.262
x10	0.0549	0.009	6.040	0.000	0.037	0.073
x11	1.5515	0.060	25.977	0.000	1.434	1.669
x12	2.1635	0.164	13.197	0.000	1.842	2.485
x13	0.0263	0.002	14.726	0.000	0.023	0.030

Table 10: State model and P-value

	feature	importance
12	avg_training_score	0.424033
10	KPIs_met >80%	0.419430
8	previous_year_rating	0.142481
11	awards_won?	0.012605
2	region	0.001450
0	employee_id	0.000000
1	department	0.000000
3	education	0.000000
4	gender	0.000000
5	recruitment_channel	0.000000
6	no_of_trainings	0.000000
7	age	0.000000
9	length_of_service	0.000000

Table 11: Feature Important

```
# Use GridSearch to find the best combination of model hyperparameters

decision_tree_02 = tree.DecisionTreeClassifier(max_depth = 2)

param_grid = { "criterion" : ["gini", "entropy", "log_loss"],
               "min_samples_leaf" : [2, 3, 4, 5],
               "min_samples_split" : [2, 3, 4, 5,],
               "max_depth" : [2, 3] }

grid_search_02 = GridSearchCV(decision_tree_02, param_grid = param_grid, scoring = 'accuracy', cv = 5, n_jobs = -1)
# cv: number of partitions for cross validation
# n_jobs: number of jobs to run in parallel, -1 means using all processors

grid_search_02 = grid_search_02.fit(X_train_02, y_train_02) #

print(grid_search_02.best_score_)
print(grid_search_02.best_params_)
```

```
0.738026013771997
{'criterion': 'entropy', 'max_depth': 3, 'min_samples_leaf': 2, 'min_samples_split': 2}
```

```
# Create Decision Tree using the best hyperparameters
decision_tree_02 = tree.DecisionTreeClassifier(criterion = 'entropy', max_depth = 3, min_samples_leaf = 2, min_samples_split = 2, random_state =

decision_tree_02.fit(X_train_02, y_train_02)
```

```
DecisionTreeClassifier(criterion='entropy', max_depth=3, min_samples_leaf=2,
                       random_state=23)
```

Table 12: Grid Search

This is the sample from (table 10-12) what technique I use to improve model accuracy. But I need to clarify one thing was for logistic regression I done p-value with different step approaches which I will explain in another part.
Same as the pre-check:

3.3.2 Post-Check Accuracy Run

- Normal Accuracy (P-Value)
- Confusion Matrix (P-Value)
  - K-Fold (P-Valve)

Post-Check Train and Test accuracy result

Stats Model	Logistic Regression
Train	0.7178270849273145
Test	0.7200999642984648

Confusion Matrix: Train	Logistic Regression
Accuracy	0.7178270849273145
Precision	0.7136739063933925
Recall	0.7198395556926874
F1	0.7167434715821812

Confusion Matrix: Test	Logistic Regression
Accuracy	0.7200999642984648
Precision	0.7240418118466899
Recall	0.7281009110021023
F1	0.7260656883298393

K-Fold: Avg	Logistic Regression
Train	0.7200086334352676
Test	0.7188311814921395

Table 13: Sample of Post-check P-Value result

Logistic Regression uses Original train test split

Remarks: Without Stats Model improvement of P-value!

GridSearch	Logistic Regression	Decision Tree	Random Forest	XGBoost	SVC
Train	0.7293037490436113	0.7387911247130834	0.7576128538638103	0.858301453710788	0.512318286151492
Test	0.7286683327383078	0.7336665476615495	0.759014637629418	0.8111388789717958	0.49018207782934664

Confusion Matrix: Train	Logistic Regression	Decision Tree	Random Forest	XGBoost	SVC
Accuracy	0.7293037490436113	0.7387911247130834	0.7576128538638103	0.858301453710788	0.512318286151492
Precision	0.7191780821917808	0.7004704652378463	0.716149230367858	0.8067850516830108	0.5257142857142857
Recall	0.7451403887688985	0.8269052761493366	0.8469608145634063	0.9392162912681271	0.1703178031471768
F1	0.7319290801636612	0.7584547898684024	0.7760814249363868	0.8679783290561733	0.2572826846888837

Confusion Matrix: Test	Logistic Regression	Decision Tree	Random Forest	XGBoost	SVC
Accuracy	0.7286683327383078	0.7336665476615495	0.759014637629418	0.8111388789717958	0.49018207782934664
Precision	0.7311157311157311	0.7008849557522124	0.7227488151658767	0.7669441141498217	0.4988814317673378
Recall	0.7393132445690259	0.8325157673440785	0.8549404344779257	0.9039943938332166	0.15627189908899788
F1	0.735191637630662	0.7610506085842409	0.78330658105939	0.829848825989064	0.23799359658484526

K-Fold: Avg	Logistic Regression	Decision Tree	Random Forest	XGBoost	SVC
Train	0.7295415802922747	0.7378695214635778	0.7612467651509832	0.8530687563227776	0.5072567720188634
Test	0.7265427709261156	0.7365037866058638	0.7587833428337782	0.8140531650244469	0.48907257689912365

Table 14: Sample of Grid Search results

This accuracy results for Logistic Regression doesn't use P-value. Mainly use Grid Seach to do the accuracy and see any improvement on it.

Post-Check Train and Test accuracy result

Feature Important & GridSearch	Decision Tree	Random Forest	XGBoost	SVC
Train	0.7387911247130834	0.7592960979342005	0.8254016832440704	"Not Applicable"
Test	0.7336665476615495	0.7611567297393788	0.8072117101035344	"Not Applicable"

Confusion Matrix: Train	Decision Tree	Random Forest	XGBoost	SVC
Accuracy	0.7387911247130834	0.7592960979342005	0.8254016832440704	"Not Applicable"
Precision	0.7004704652378463	0.7112462006079028	0.766092245311708	"Not Applicable"
Recall	0.8269052761493366	0.8663992594878124	0.9327368096266584	"Not Applicable"
F1	0.7584547898684024	0.7811934900542495	0.8412411298177265	"Not Applicable"

Confusion Matrix: Test	Decision Tree	Random Forest	XGBoost	SVC
Accuracy	0.7336665476615495	0.7611567297393788	0.8072117101035344	"Not Applicable"
Precision	0.7008849557522124	0.7168192219679634	0.7556195965417868	"Not Applicable"
Recall	0.8325157673440785	0.8780658724597057	0.9187105816398038	"Not Applicable"
F1	0.7610506085842409	0.7892913385826772	0.8292220113851992	"Not Applicable"

K-Fold: Avg	Decision Tree	Random Forest	XGBoost	SVC
Train	0.7378695214635778	0.7607647621595793	0.8226756829496216	"Not Applicable"
Test	0.7365037866058638	0.7588904092149343	0.8140528209439506	"Not Applicable"

Table 15: Sample of Feature Important and Grid Search

This Table 15 hyperparameter focuses mainly on Decision Tree, Random Forest and XGBoost. After I performed all the necessary models and used feature selection, hyperparameter tuning, and evaluation techniques.

3.3.3 Result Pre & Post Check Accuracy Results

After performing the individual parts of models' improvement, I can strongly say that certain models have decent models upgraded.

I will start explaining individual models' performance improvements. And choose which improvements are highly recommended for training models.

Logistic Regression

For the explanation on the p-value, why I will do separate as:

P-Value Analysis

The p-value analysis was conducted separately to evaluate the impact of removing statistically insignificant variables, "department". After removing the features from the dataset, the training and testing accuracy showed a slight improvement compared to the pre-run model. Similar improvements were observed in the confusion matrix results for both training and testing, as well as in k-fold cross-validation performance.

Grid Search (Logistic Regression without P-Value)

In this approach, no p-value–based feature removal was applied. Grid Search outperformed the p-value method in terms of overall performance. Improvements were observed across confusion matrix metrics for both training and testing datasets, and k-fold cross-validation results also indicated better model stability and accuracy.

Feature Importance & Grid Search

Logistic Regression does not provide traditional feature importance like tree-based models; instead, feature relevance is interpreted through coefficient values. Logistic Regression does support Grid Search, and the best-performing Logistic Regression model was achieved using Grid Search without applying p-value filtering. This approach resulted in the most effective improvement for Logistic Regression.

Decision Tree

Grid Search

Comparing the pre-run and post-run models, Grid Search significantly improved Decision Tree's performance. Hyperparameter tuning helped control model complexity, reduce overfitting, and improve overall predictive performance.

Feature Importance & Grid Search

The feature importance results were consistent with the Grid Search outcomes, showing no major changes in feature rankings. The post-check model using Grid Search achieved the best performance, making it the most effective improvement strategy for the Decision Tree.

Random Forest

Grid Search

For Random Forest, the pre-check model performed better than the Grid Search–tuned model. This suggests that the default parameters were already well-suited to the dataset, and additional hyperparameter tuning did not result in significant performance gains.

Feature Importance & Grid Search

Similar to the Grid Search results, the pre-check model outperformed the tuned versions. This conclusion was supported by confusion matrix metrics for training and testing, as well as k-fold cross-validation results. Therefore, Random Forest did not benefit substantially from further model improvement techniques.

XGBoost

Grid Search

When comparing pre-run and post-run models, some performance metrics improved while others declined. With Grid Search, training performance improved; however, testing accuracy did not show consistent improvement. The confusion matrix indicated higher precision but lower recall, resulting in mixed F1-score outcomes. For the testing set, precision increased, while recall and F1-score decreased. Among all approaches, the pre-check k-fold cross-validation produced the most stable results.

Feature Importance & Grid Search

The pre-run XGBoost model achieved the best overall performance, indicating that additional feature removal or tuning was unnecessary for this model.

SVC

Grid Search

Grid Search led to some performance improvement; however, confusion matrix results revealed weaknesses in recall or F1-score, despite precision performing well. K-fold cross-validation showed slight improvement in training performance, but testing performance improved only marginally.

Feature Importance & Grid Search

Feature importance is not applicable to SVC, as it does not provide inherent feature importance

measures.

3.4 Final Summary

3.4.1 Model Ranking (Best to Worst)

1. XGBoost (Best performance in both pre-check results and post-check Grid Search results)
2. Random Forest (Pre-Check Performance)
3. Decision Tree (Post-Check result Performance)
4. Logistic Regression (Post-Check Grid Search)
5. SVC (Post-Check Grid Search)

Although several models showed improvements after post-check evaluation, XGBoost consistently outperformed all other models regardless of whether pre-check or post-check techniques were applied. This demonstrates that XGBoost remains highly effective even without extensive tuning.

In comparison, Random Forest achieved its best performance during the pre-check stage, indicating that its default parameters were already well suited to the dataset and that additional tuning provided limited improvement.

3.4.2 Final Selection

XGBoost post-check Grid Search, was selected as the final model due to its superior accuracy, robustness, and consistency across evaluation techniques. An accuracy of approximately 82% indicates that the model correctly classifies employee promotion outcomes in the majority of cases. Therefore, the XGBoost model serves as a reliable decision-support tool for identifying potential promotion candidates, while final promotion decisions remain the responsibility of HR professionals.

4.0 AIRBNB

4.1 Problem understanding and the approaches

4.4.1 Regression Problem

This project focuses on a regression problem using an Airbnb dataset, where the target variable (y) is the price of a listing. Different rooms have different price ranges, which can also depend on how long guests stay overnight.

Typically, travelers consider several key factors when choosing an Airbnb, including:

1. Number of reviews
2. Room type
3. Neighborhood or district
4. Availability of days
5. Price

Machine learning will be applied to predict Airbnb prices based on these features, allowing us to analyze price variations throughout the year and address the regression problem effectively.

4.4.2 Handling Missing Values

The dataset is mostly clean; however, the feature “reviews_per_month” contains missing values (NaN) that need to be handled before training the models.

```
# count how many null value -> sum()
# count how many null value -> mean() * 100
nan_sum_02 = data_04.isnull().sum()
nan_percentage_02 = data_04.isnull().mean() * 100
```

```
print(nan_sum_02)
```

```
id                                0
host_id                           0
neighbourhood_group               0
latitude                          0
longitude                         0
room_type                         0
price                             0
minimum_nights                    0
number_of_reviews                 0
last_review                       0
reviews_per_month              2485
calculated_host_listings_count    0
availability_365                  0
dtype: int64
```

```
print(nan_percentage_02)
```

```
id                                0.000000
host_id                           0.000000
neighbourhood_group               0.000000
latitude                          0.000000
longitude                         0.000000
room_type                         0.000000
price                             0.000000
minimum_nights                    0.000000
number_of_reviews                 0.000000
last_review                       0.000000
reviews_per_month                34.399225
calculated_host_listings_count    0.000000
availability_365                  0.000000
dtype: float64
```

Table 1: NaN value of “reviews_per_month”

Table 1 show the number of NaN values in the “reviews_per_month” column.
This feature represents the number of reviews for each rental unit and is important for analyzing rental performance. Without properly handling the missing values, the model’s ability to predict price accurately and understand customer preferences may be compromised.

Therefore, imputation is required to solve the missing value. Because this variable is important for renting performance analysis, I imputed the missing values using statistical methods such as mean, median, and mode.

```
## need to remove or imputation?

checking_value = data_05['reviews_per_month'].value_counts().sort_index(ascending = False)
print(checking_value)

def explanation():
    """
Upon seeing the result:
It is not suitable to performed mode!

So instead I will perform mean or median.
    """

print(explanation.__doc__)
```

```
reviews_per_month
13.00      1
12.60      1
12.00      1
11.03      1
8.37       1
         ...
0.05      86
0.04     100
0.03      70
0.02      59
0.01       1
Name: count, Length: 510, dtype: int64
```

Table 2: “reviews_per_month”, value_count()

Table 2: I used value_count() to observe the behavior, and it is not suitable to do mode. As the value has too much and is best to do mean and median.

```
# Convert the 'reviews_per_month' column to numeric values.
# 'errors="coerce"' median: if any value cannot be converted to a number (like a string or invalid entry),
# it will be replaced with NaN. This avoids errors and ensures the column is fully numeric.
data_05['reviews_per_month'] = pd.to_numeric(data_05['reviews_per_month'], errors='coerce')

# Fill any NaN values (including ones created by coerce) with the median of the column.
data_05['reviews_per_month'] = data_05['reviews_per_month'].fillna(data_05['reviews_per_month'].median())
```

```
data_05.head()
```

	id	host_id	neighbourhood_group	latitude	longitude	room_type	price	minimum_nights	number_of_reviews	last_review	reviews_per_month	calcula
0	49091	266763	5	1.44255	103.79580	0	83	180	1	2136.0	0.01	
1	50646	227796	1	1.33235	103.78521	0	81	90	18	1705.0	0.28	
2	56334	266763	5	1.44246	103.79667	0	69	6	20	1426.0	0.20	
3	71609	367042	3	1.34541	103.95712	0	206	1	14	16.0	0.15	
4	71896	367042	3	1.34567	103.95963	0	94	1	22	30.0	0.22	

Table 3: Median

```
# Convert the 'reviews_per_month' column to numeric values.
# 'errors="coerce"' means: if any value cannot be converted to a number (like a string or invalid entry),
# it will be replaced with NaN. This avoids errors and ensures the column is fully numeric.
data_05['reviews_per_month'] = pd.to_numeric(data_05['reviews_per_month'], errors='coerce')

# Fill any NaN values (including ones created by coerce) with the mean of the column.
data_05['reviews_per_month'] = data_05['reviews_per_month'].fillna(data_05['reviews_per_month'].mean())
```

```
data_05.head()
```

	id	host_id	neighbourhood_group	latitude	longitude	room_type	price	minimum_nights	number_of_reviews	last_review	reviews_per_month	calcula
0	49091	266763	5	1.44255	103.79580	0	83	180	1	2136.0	0.01	
1	50646	227796	1	1.33235	103.78521	0	81	90	18	1705.0	0.28	
2	56334	266763	5	1.44246	103.79667	0	69	6	20	1426.0	0.20	
3	71609	367042	3	1.34541	103.95712	0	206	1	14	16.0	0.15	
4	71896	367042	3	1.34567	103.95963	0	94	1	22	30.0	0.22	

Table 4: Mean

Tables 3 and 4 show that both the mean and median produced similar values. Since the mode contained many outlier values (as seen in Table 2), mean imputation was chosen to fill the missing values in the "reviews_per_month" feature.

4.2Build the model(s)

4.2.1 Model Selection and Training Approach

After completing data preprocessing, including handling missing values, I performed a **train-test split** to evaluate the performance of multiple regression models.

```
# Split both Inputs (X) and Ouput (y) into training set (70%) and testing set (30%)
X_train_05, X_test_05, y_train_05, y_test_05 = train_test_split(X_03, y_03, test_size = 0.3, random_state = 55)
```

Table 5: Sample of train-test split for regression models

4.2.2 Machine Learning Models Used

Since this is a regression task, the following four algorithms were applied:

- Linear Regression
- Decision Tree Regression
- Random Forest Regression
- XGBoost Regression

The Support Vector Regressor (SVR) was not used because it was inefficient for this dataset, taking too long to run and not providing practical performance benefits. Therefore, only the four models above were trained and compared to determine the best-performing model for predicting Airbnb rental prices.

In addition, pre-check results were performed to compare with post-check results to ensure consistency and stability in evaluation.

4.2.3 Evaluation Techniques

- K-Fold Cross Validation

So below is the sample of the result of individual technique used:

```
The Linear Regression Train R^2 Value is: 0.2009511361357118
The Linear Regression Train MAE value is: 50.70225094347705
The Linear Regression Train MSE value is: 4316.322823118184
The Linear Regression Train RMSE is: 65.69872771308577
```

```
The Linear Regression, K-Fold train R^2 average result is: 0.20161640610456472
The Linear Regression, K-Fold train MAE average result is: 50.67700672464135
The Linear Regression, K-Fold train MSE average result is: 4312.714442530847
The Linear Regression, K-Fold train RMSE average result is: 65.6698025067889
```

Table 6: Sample of Pre-check Linear Regression Matrix

Table 6 shows the evaluation metrics of the Linear Regression model to assess whether the overall model performance is satisfactory.

2.3 Pre & Post-Check Train and Test R^2, MAE, MSE, RMSE result

Pre-Check Train and Test R^2, MAE, MSE & RMSE results

Train	Linear Regression	Decision Tree	Random Forest	XGBoost
Train R^2	0.2009511361357118	0.44375595262879197	0.5841918817258428	0.7734876871109009
Train MAE	50.70225094347705	42.418147228433476	36.563819808397405	25.979721069335938
Train MSE	4316.322823118184	3004.7334843589188	2246.1230496779676	1223.5802001953125
Train RMSE	65.69872771308577	54.815449321873835	47.39328063848258	34.97971126517931

Test	Linear Regression	Decision Tree	Random Forest	XGBoost
Test R^2	0.18900244576439307	0.4715616375215118	0.5719310520545546	0.6582671403884888
Test MAE	50.28543352632267	40.88791356259632	36.3044964331423	31.27280044555664
Test MSE	4225.28854476308	2753.1581913316754	2230.234619915037	1780.42431640625
Test RMSE	65.00221953720565	52.47054594085786	47.225359923615585	42.195074551495345

Train K-Fold	Linear Regression	Decision Tree	Random Forest	XGBoost
Train R^2	0.20161640610456472	0.4438422059238382	0.5837304290770927	0.7828145146369934
Train MAE	50.67700672464135	42.41239212904535	36.37497121487026	25.452125549316406
Train MSE	4312.714442530847	3004.1079596403265	2248.292138173197	1173.053662109375
Train RMSE	65.6698025067889	54.80830995190621	47.415039175940315	34.246888335855324

Test K-Fold	Linear Regression	Decision Tree	Random Forest	XGBoost
Test R^2	0.19485910991811142	0.4422389094553809	0.544222499231225	0.6415774106979371
Test MAE	50.90547827742526	42.46189001314684	37.98407695425868	32.72531509399414
Test MSE	4349.088850558044	3010.4712088708607	2458.767209577117	1932.3084716796875
Test RMSE	65.92369119797874	54.844190325606654	49.56802629213819	43.94127036329678

Table 7: Model Scoring Performance

4.2.4 Normal Pre-Check Regression evaluation metrics

In Table 7, the following metrics were used to evaluate model performance:

- R^2 (Coefficient of Determination) -> Higher the better
- MAE (Mean Absolute Error) -> Lower the better
- MSE (Mean Squared Error) -> Lower the better
- RMSE (Root Mean Squared Error) -> Lower the better

Which has the highest R^2 and better evaluation result on train test?

1. Random Forest (Very stable, less overfitting)
2. Decision Tree (Very Stable, lower max performance, gap in-between)
3. Linear Regression (Stable but poor predictive power, gap in-between)
4. XGBoost (Slight overfitting, but test R^2 still highest, best predictive power)

Which has the lowest MAE and better evaluation result on train test?

1. Random Forest (Very small gap, stable, good performance)
2. Decision Tree (Small gap, stable, moderate performance)
3. Linear Regression (Very small gap, stable, but MAE is high, poor predictive)
4. XGBoost (Larger gap, slight overfitting, but lowest test MAE, best predictive)

Which has the lowest MSE and better evaluation result on train test?

1. Random Forest (Very small gap, stable, good performance)
2. Decision Tree (Small gap, stable, moderate performance)
3. Linear Regression (Very small gap, stable, but MSE is high, poor predictive)
4. XGBoost (Larger gap, train and test overfitting, but lowest test MSE, best predictive)

Which has the lowest RMSE and better evaluation result on train test?

1. Random Forest (Very small gap, stable, good performance)
2. Decision Tree (Small gap, stable, moderate performance)
3. Linear Regression (Very small gap, stable, but RMSE is high, poor predictive)
4. XGBoost (Larger gap, train and test overfitting, but lowest test RMSE, best predictive)

4.2.5 Pre-Check Evaluation Result for Model Performance

XGBoost is the best model for future predictions, as it achieves the highest test performance across all evaluation metrics. If XGBoost is not used, the next best alternative is Random Forest, which provides slightly lower predictive accuracy but demonstrates good stability.

4.3 Evaluate and improve the model(s)

These methods help improve the model and understand which features matter the most.

Feature selection

- P-value (Linear Regression)
- Feature Important (Tree-based models)

Hyperparameter Tuning

- Grid Search

Evaluation Techniques

- K-fold Cross validation

OLS Regression Results

Dep. Variable:	y	**R-squared (uncentered):**	0.795
Model:	OLS	**Adj. R-squared (uncentered):**	0.794
Method:	Least Squares	**F-statistic:**	1626.
Date:	Fri, 19 Dec 2025	**Prob (F-statistic):**	0.00
Time:	15:31:56	**Log-Likelihood:**	-28337.
No. Observations:	5056	**AIC:**	5.670e+04
Df Residuals:	5044	**BIC:**	5.678e+04
Df Model:	12		
Covariance Type:	nonrobust		

Table 8: State Model

	coef	std err	t	P>|t|	[0.025	0.975]
x1	-1.348e-08	1.31e-07	-0.103	0.918	-2.71e-07	2.44e-07
x2	7.417e-08	1.38e-08	5.379	0.000	4.71e-08	1.01e-07
x3	-3.9638	1.497	-2.647	0.008	-6.899	-1.029
x4	-163.5806	47.504	-3.444	0.001	-256.709	-70.453
x5	2.9122	0.587	4.958	0.000	1.761	4.064
x6	35.8062	1.686	21.235	0.000	32.500	39.112
x7	-0.1438	0.024	-6.093	0.000	-0.190	-0.098
x8	-0.0778	0.048	-1.608	0.108	-0.173	0.017
x9	0.0072	0.001	5.800	0.000	0.005	0.010
x10	3.7910	1.331	2.848	0.004	1.181	6.401
x11	0.1736	0.016	10.588	0.000	0.141	0.206
x12	0.0156	0.007	2.317	0.021	0.002	0.029

Omnibus:	129.987	**Durbin-Watson:**	1.981
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	141.858
Skew:	0.381	**Prob(JB):**	1.57e-31
Kurtosis:	3.305	**Cond. No.**	6.34e+09

	feature	importance
5	room_type	0.908988
11	availability_365	0.044864
6	minimum_nights	0.031995
10	calculated_host_listings_count	0.007775
3	latitude	0.006378
0	id	0.000000
1	host_id	0.000000
2	neighbourhood_group	0.000000
4	longitude	0.000000
7	number_of_reviews	0.000000
8	last_review	0.000000
9	reviews_per_month	0.000000

Table 9: P-Value & Feature Important

From the (table 8-9) I remove the value that either is high value or low value due to the p-value and feature important differences. After I removed the value, I do a post-check on every individual model from the pre-check models. And for linear regression, it doesn't support features important.

So, the individual model can select using feature selection, hyperparameter tuning or evaluation technique. And once I did everything and ran the model with the regression evaluation.

4.3.1 Post-Check Result Evaluation

Post-Check Train and Test R^2, MAE, MSE & RMSE results

Train Stats Model	Linear Regression
Train R^2	0.20038603454720838
Train MAE	50.723235990509714
Train MSE	4319.375403497364
Train RMSE	65.72195526228175

Test Stats Model	Linear Regression
Test R^2	0.18878722249197843
Test MAE	50.277854648647825
Test MSE	4226.409855700148
Test RMSE	65.01084413926763

Train K-Fold	Linear Regression
Train R^2	0.20093406010922052
Train MAE	50.70369645680106
Train MSE	4316.40237838512
Train RMSE	65.6978557108355

Test K-Fold	Linear Regression
Test R^2	0.19538728837715444
Test MAE	50.89101610601291
Test MSE	4346.3549224034305
Test RMSE	65.90231045270848

Table 10: Linear regression p-value result

Post-Check GridSearch Train and Test R^2, MAE, MSE & RMSE results

Train	Decision Tree	Random Forest	XGBoost
Train R^2	0.47158764947710663	0.48345061225701347	0.8267752528190613
Train MAE	41.30142834463542	40.868965770347984	22.504976272583008
Train MSE	2854.3915043559405	2790.3098451328947	935.7300415039062
Train RMSE	53.42650563489943	52.82338350705012	30.58970482864956

Test	Decision Tree	Random Forest	XGBoost
Test R^2	0.4927237570806261	0.506723571958176	0.6689131855964661
Test MAE	40.15462420942888	39.4816961595043	30.375141143798828
Test MSE	2642.9037757800656	2569.96489067249	1724.9586181640625
Test RMSE	51.409179878500936	50.69482114252392	41.53262113284042

Train K-Fold	Decision Tree	Random Forest	XGBoost
Train R^2	0.47294144781515507	0.4853175589711386	0.846998143196106
Train MAE	41.261653259352315	40.796914183818764	21.182170104980468
Train MSE	2846.8017403611207	2779.99181412702	826.339111328125
Train RMSE	53.354344149029274	52.72464923720905	28.745358111756467

Test K-Fold	Decision Tree	Random Forest	XGBoost
Test R^2	0.4609021035894788	0.4733105086523815	0.664033317565918
Test MAE	41.66546842581032	41.19742890630799	31.25770149230957
Test MSE	2907.461293521274	2841.380629826498	1811.483447265625
Test RMSE	53.90341280163125	53.2854662823766	42.544688330490416

Post-Check Feature Important & GridSearch Train & Test R^2, MAE, MSE & RMSE results

Train	Decision Tree	Random Forest	XGBoost
Train R^2	0.47158764947710663	0.48178283764935614	0.8211401104927063
Train MAE	41.30142834463542	40.92239571978894	22.602008819580078
Train MSE	2854.3915043559405	2799.318873151575	966.169921875
Train RMSE	53.42650563489943	52.90858978607893	31.083273988996076

Test	Decision Tree	Random Forest	XGBoost
Test R^2	0.4927237570806261	0.5048483377188923	0.6552649736404419
Test MAE	40.15462420942888	39.572598287879934	30.703266143798828
Test MSE	2642.9037757800656	2579.7348409129213	1796.065673828125
Test RMSE	51.409179878500936	50.79109017251866	42.3800150286444

Train K-Fold	Decision Tree	Random Forest	XGBoost
Train R^2	0.47294144781515507	0.48348350807418905	0.8385046005249024
Train MAE	41.261653259352315	40.870332315723054	21.581455993652344
Train MSE	2846.8017403611207	2789.892943218504	872.2520751953125
Train RMSE	53.354344149029274	52.81850577162375	29.53323955320685

Test K-Fold	Decision Tree	Random Forest	XGBoost
Test R^2	0.4609021035894788	0.4723461974060384	0.6469622373580932
Test MAE	41.66546842581032	41.25208994890212	31.69563980102539
Test MSE	2907.461293521274	2846.48983438134	1902.9452392578125
Test RMSE	53.90341280163125	53.33336821108939	43.604762754908734

Table 11: Grid Search and Feature Important

4.3.2 Result Pre & Post Check Evaluation Matrix Results

After doing the post-check on all the models. Certainly, models improve and certainly downgraded.

I will start by explaining individual model performance and the final will decide the Airbnb renting price.

4.2.3 Post-Check Regression evaluation metrics

In Table 11, the following metrics were used to evaluate model performance:

- R^2 (Coefficient of Determination) -> Higher the better
- MAE (Mean Absolute Error) -> Lower the better
- MSE (Mean Squared Error) -> Lower the better
- RMSE (Root Mean Squared Error) -> Lower the better

Linear Regression

P-Value Analysis

I removed the features id and number_of_reviews from the Airbnb dataset because their p-values were higher than 0.05, indicating they are not statistically significant.

- Train performance: After removal, the R^2 on the training set did not improve significantly, and MAE, MSE, and RMSE were slightly higher, which is acceptable for training.
- Test performance: On the test set, R^2 also did not improve noticeably, but MAE, MSE, and RMSE showed slight improvement, suggesting a minor benefit in predictive error.
- K-Fold cross-validation: In the pre-check evaluation, the test R^2 was slightly higher than the training R^2, indicating better generalization. However, in the post-check evaluation, the test MAE, MSE, and RMSE did not show significant improvement, suggesting that removing these features did not substantially enhance model performance across all metrics.

Grid Search

Linear Regression does not need Grid Search, because the standard model has no parameters to tune.

Feature Importance & Grid Search

Linear Regression does not provide traditional feature importance like tree-based models. Instead, the model coefficients can be used to understand the effect of each feature on the target.

Decision Tree

Grid Search

For Decision Tree after post-training with Grid Search, the train R^2 increased and MAE, MSE, and RMSE decreased, showing better performance.

On the test set, all metrics also improved compared to the pre-check results, indicating better overall model performance.

The K-Fold results in the post-check also showed noticeable improvement, demonstrating that Decision Tree benefits from hyperparameter tuning as part of the evaluation process.

Feature Importance & Grid Search

The feature importance results were consistent with the Grid Search outcomes, showing no significant change in feature rankings. The post-check model under Grid Search demonstrated the best overall performance, making it the most effective improvement strategy for the Decision Tree.

Random Forest

Grid Search

For Random Forest, the pre-check model performed better than the Grid Search–Hyperparameter tuned model. This suggests that the default parameters were already well-suited to the dataset, and further tuning did not yield meaningful improvements.

Feature Importance & Grid Search

Feature importance analysis combined with Grid Search generally showed little to no improvement. Overall, the Random Forest pre-check model remained the best-performing version, indicating that this model did not benefit significantly from additional tuning.

XGBoost

Grid Search

For pre-run and post-run check, Grid search and inclusive with k-fold was the best choice for XGBoost to have hyperparameter tuning and elevation technique.

Feature Importance & Grid Search

For Feature Importance and Grid Search, the train metrics improved noticeably. However, on the test set, R^2 decreased slightly, MAE decreased a little, and MSE and RMSE increased slightly, indicating minor overfitting after tuning. The K-Fold results showed better performance than the earlier results. This demonstrates that to achieve reliable improvements without unintended increases or decreases in performance, Feature Importance and Grid Search should be applied together with proper evaluation techniques.

4.4 Final Summary

4.4.1 Model Ranking (Best to Worst)

1. XGBoost (Best performance in both pre-check results and post-check Grid Search results)
2. Random Forest (Pre-Check Performance)
3. Decision Tree (Post-Check result Performance)
4. Linear Regression (Pre-Check)

Although several models improved after post-check evaluation, XGBoost consistently outperformed all other models, demonstrating strong predictive performance even without extensive tuning. Random Forest achieved its best performance during the pre-check stage, indicating that its default parameters were already well suited to the dataset, and additional tuning provided limited improvement.

4.4.2 Final Selection:

XGBoost (pre-check or post-check Grid Search) was selected as the final model due to its superior evaluation metrics compared to other tested models. If needed, the pre-check result can be used directly, requiring less effort, while post-check techniques like hyperparameter tuning, feature selection, or evaluation strategies can be applied to further refine the model.

This regression model uses XGBoost to predict the price target. The model's performance is evaluated using test results to determine whether it can make reliable predictions on future, unseen data.

5.0 Conclusion

5.1 Summarize your work on these two problems

This assignment on HR Analytics required a significant amount of time, as five different models were implemented and evaluated. Although fewer models could have been used, multiple models were tested to gain a deeper understanding of how each algorithm performs and to compare their strengths and weaknesses. Experimenting with different approaches provided useful insights into individual model performance. While the process involved some trial and error, it helped clarify how evaluation methods, such as the confusion matrix, indicate whether a classification model performs well. Overall, this assignment improved my understanding of model selection, evaluation, and performance comparison in machine learning.

The Airbnb price prediction task required considerable time, as multiple regression models were evaluated. Among these models, Support Vector Regression (SVR) was particularly time-consuming due to its high computational cost and longer training time. Although SVR is suitable for regression problems with continuous targets such as price, its inefficiency made it less practical for this dataset. As a result, SVR was excluded, and the remaining four regression models were used to evaluate performance. This process highlighted the importance of selecting models that are both appropriate for the problem and computationally efficient. When applicable, techniques such as hyperparameter tuning, feature importance analysis, and evaluation metrics were applied to better understand model behavior and improve performance.

Across both projects, XGBoost consistently performed at the top, followed by Random Forest. Testing XGBoost confirmed that it is both highly efficient and the best-performing model, outperforming all other models in terms of evaluation metrics. If XGBoost cannot be selected, Random Forest using the pre-check results could be considered; however, additional work may be required to improve test performance and ensure better predictive reliability.

6.0 Reflection

6.1 Suggest possible further improvement(s) to the current solution.

For the HR Analytics task, I am uncertain whether the imputation method I applied was appropriate.

previous_year_rating	length_of_service
5.0	3
5.0	6
4.0	17
	1
	1
3.0	7
5.0	7
4.0	2
3.0	3
	1

Table 1: HR Analytics: Sample of “previous_year_rating” and “length_of_service”

After reviewing the dataset, particularly the features “previous_year_rating” and “length_of_service”, I noticed that the missing values in “previous_year_rating” appeared to be closely related to “length_of_service”. Specifically, the missing ratings occurred mainly when “length_of_service” was equal to 1. This suggests that the missing values may not be random, and using mode imputation may not have been suitable. This issue indicates a limitation in my data preprocessing approach, and further investigation is required to determine a more appropriate imputation strategy in future work.

number_of_reviews	last_review	reviews_per_month
41	160.0	0.66
2	1886.0	0.03
10	451.0	0.16
0	1887.0	
43	75.0	0.7
0	1887.0	
22	106.0	0.35
0	1887.0	
154	31.0	2.99
0	1887.0	
179	15.0	2.87
118	25.0	2.67
137	43.0	2.25
80	290.0	1.3
2	1053.0	0.04
18	1011.0	0.29
41	14.0	0.69
0	1887.0	

Table 2: Airbnb: Sample of “number_of_reviews” and “reviews_per_month”

In the Airbnb price prediction task, an error was made when imputation the “reviews_per_month” feature using the mean. This mistake occurred because I did not initially recognize that “reviews_per_month” is directly linked to “number_of_reviews”. As explained in the lecture, when “number_of_reviews” is zero, “reviews_per_month” should also be zero rather than imputed using statistical measures such as mean, median, or mode. This oversight led to incorrect data representation and affected the overall evaluation results.

As shown in Table 2, the NaN values in “reviews_per_month” correspond to rows where “number_of_reviews” equals zero. Identifying this relationship was a valuable learning experience, as it highlighted the importance of understanding feature dependencies during data cleaning. Due to this

issue, the evaluation results for the Airbnb models were affected. In future practice, I will place greater emphasis on exploratory data analysis and feature relationships to ensure accurate data preprocessing.

Additionally, while Linear Regression and Logistic Regression models can still be applied, they may not be fully reliable for complex datasets. In this project, tree-based models such as Decision Tree, Random Forest, and XGBoost demonstrated significantly better performance. Future improvements may involve prioritizing these models and further optimizing them to achieve better predictive accuracy and more reliable evaluation metrics (RMSE, MSE, MAE, R^2).

6.2 With reference to the module leaning objectives stated, reflect on the skills learnt and the skills you could have learnt better.

From this practice, one of the most important skills I learned was working with Random Forest, as it performed very well during the pre-train-test split evaluation. However, I realized that I did not fully explore its potential during the post-train-test split phase, and there is room to improve my use of this model in future projects.

Another key skill I developed is data cleaning and preprocessing. I learned that careful handling of missing values, feature relationships, and exploratory data analysis is crucial to ensure accurate and reliable model performance. In future practice, I aim to strengthen these skills further by systematically analyzing data dependencies and applying appropriate preprocessing techniques.